



where is the PRD?

You're right — I hadn't delivered it as a document yet. Below is the PRD, incorporating the existing kitchen systems so it extends rather than duplicates them: the back-wall queue board, timed cooking, station menu, visual-state language, and kitchen-command focus system. ^[1] ^[2] ^[3] ^[4]

Rival Restaurant

Kitchen Operations Fidelity PRD

Document status: Proposed

Feature area: Co-op kitchen gameplay, kitchen readability, prep planning, inventory clarity

Target modes: Co-op first; presentation improvements may benefit solo and bot modes

Primary outcome: Make the kitchen a readable, strategic, cooperative workspace where players can assess demand, choose what to cook, start asynchronous production, pre-prep for expected demand, and react to shortages without excessive panel navigation.

Product summary

Co-op players already manage a restaurant together, but kitchen interactions need more operational fidelity and better in-world readability. The game should make cooking meaningful through station selection, cook duration, capacity, demand awareness, and pre-prep decisions —not through repetitive dexterity minigames.

The existing kitchen queue display has already evolved from small 3D food proxies into a larger, live back-wall surface with two-dimensional dish cards. However, the kitchen must now become easier to operate while moving through the world: entering the kitchen should give players a closer operational camera, stations should make cooking intent clear, inventory risk should be actionable immediately, and players should be free to cook dishes speculatively ahead of expected demand. ^[1]

(see the generated image above)

Goals

- Make the kitchen queue and station state legible during active play.
- Give players a purposeful kitchen camera state when they enter the back-of-house area.

- Preserve asynchronous, time-based cooking rather than introducing chopping, timing, or reflex minigames.
- Let players start any valid dish at the appropriate station, including dishes not yet ordered.
- Make low inventory visible before it becomes a full service failure.
- Provide a quick path to restock critical ingredients without hiding it behind a management panel.
- Support intentional prep ahead of promotions, events, sales, and rushes.
- Preserve server-authoritative simulation, existing interaction validation, and the current snapshot-driven scene architecture.^[5]

Non-goals

- No manual chopping, flipping, stirring, temperature control, or rhythm-game cooking.
- No local-only inventory, cook completion, station state, or ticket prioritization logic.
- No replacement for the existing kitchen command system and its policies such as Rush the Pass or Premium First.
- No replacement for the full pantry/restock management surface; this adds a fast operational path.
- No automatic correctness requirement: players may deliberately cook a dish with no active demand.
- No new visual system that duplicates the existing fixed state-color language.^{[2] [6]}

Design principles

Operational clarity over UI density

A kitchen player should understand, within a few seconds:

- What dishes are needed now
- Which station is waiting, active, blocked, or ready
- What food is already prepared
- Which ingredients are at risk
- Whether a quick restock is available
- What work will produce the most immediate service benefit

The scene must carry this information at a glance, with detailed panels serving as optional drill-down tools. This follows the existing visual-state approach: gameplay state is published by the authoritative simulation and rendered by Three.js and React surfaces without the view layer inventing game state.^{[6] [5]}

Strategic cooking, not mechanical cooking

Cooking should ask players to decide **what**, **where**, and **when** to cook. Once started, an item runs under the normal station timer; the player can leave to work elsewhere. This mirrors the game's existing timed station behavior, in which player-started work already uses the same authoritative kitchen timer path as worker-started work.^[4]

One source of truth

The queue board, station menus, kitchen zoom HUD, quick-restock surfaces, and co-op teammate views must all render the same server-authoritative match snapshot. Existing ordering, station capacity, ingredient claims, and action validation remain owned by the server.^{[7] [5]}

Player experience

1. A player enters the kitchen zone.
2. The camera smoothly transitions to a closer, clearer kitchen-operations framing.
3. The back-wall queue board becomes immediately readable, showing priority dishes as large picture cards.
4. The player sees station state and ingredient risk directly in the scene.
5. At a station, the player opens a dish-selection surface and chooses what to cook.
6. The selected dish starts an authoritative cook timer; the player leaves or begins another task.
7. A quick-restock control appears when critical stock is low, with clear cost and outcome feedback.
8. The player may prep dishes before an order exists, trading station capacity and potential hold quality for faster future fulfillment.
9. The same live state is visible to the co-op partner, preventing ambiguity about what is currently being prepared.

Story 1: Kitchen operations camera

User story

As a player who walks into the kitchen, I want the camera to move into a clear kitchen-operations view so that I can read the queue board, evaluate station state, and make cooking decisions without losing the broader restaurant context.

Requirements

- Entering the configured kitchen zone activates a contextual kitchen camera state.
- The camera transitions smoothly from the default restaurant view to a closer, lower, or more kitchen-oriented framing.
- The view must keep the back-wall order board, active stations, and nearby interactable surfaces readable at once.
- The transition must not alter player authority, movement, simulation state, or the other co-op player's camera.
- Leaving the kitchen zone smoothly restores the default camera framing.
- Reduced-motion settings must replace the camera interpolation with an immediate but non-disorienting transition.
- The camera must not clip through scene geometry or obscure critical station interactions.
- The view should preserve visual awareness of the pass or dining room enough for a kitchen player to react to ready-food and urgency signals.

Acceptance criteria

- A player entering the kitchen sees a visibly tighter and more legible operational view.
- The queue board is readable without interacting with it or opening its text-detail panel.
- The player can still identify at least the nearest active, waiting, blocked, and ready stations.
- Camera transitions do not disrupt interaction range checks or cooking actions.
- Co-op clients independently control their own kitchen camera state.
- The feature is validated in the restaurant-layout and kitchen-bottleneck harnesses.

Implementation notes

The camera is client presentation only. Kitchen proximity should be derived from existing spatial layout information or a dedicated client-side kitchen-zone definition; it must not create a new server authority concept. The existing scene already separates React overlays, Three.js presentation, and server-owned match state, which this story must preserve. ^[5] ^[6]

Story 2: Kitchen queue board readability

User story

As a kitchen player, I want a large, clear, always-visible display of dishes that need to be made so that I can prioritize work without opening a panel or guessing what the restaurant needs.

Requirements

- Retain the existing queue-board data source and server-ranked priority order.
- Use large flat 2D dish-picture cards as the primary at-a-glance presentation.
- Do not show both a compact 3D dish grid and a competing 2D board for the same queue.
- Keep the current click-to-open text-detail panel as a secondary surface if it continues to provide unique information such as station, remaining time, or ingredient-blocking detail.
- Update the board live when orders arrive, priority shifts, a dish begins cooking, or a dish leaves the queue.
- Keep the queue board restaurant-private; a player must never see the rival restaurant's private queue.
- Make priority, blocked-by-stock, and station association understandable without requiring a text-heavy overlay.

Acceptance criteria

- The queue board remains readable from the kitchen camera and default camera.
- A player can distinguish top-priority dishes from lower-priority dishes.
- A player can identify a blocked dish through a visually distinct shortage or blocked state.
- The board reflects the same queue state as the authoritative order system.
- The board does not visually overlap the pass, ready-food slots, pantry, or station affordances.
- Board updates remain live and free of stale dish cards.

Existing foundation

The existing queue board already renders live queue state and has been expanded to a larger 2D-picture display. It uses the existing ranked queue data, replaces the earlier 3D proxy presentation, and remains distinct from the kitchen command board, which controls policy rather than showing the cooking queue. ^[7] ^[2] ^[1]

Story 3: Station-based dish selection

User story

As a kitchen player, I want to approach a station and choose a specific dish to start so that I can make deliberate production decisions rather than merely reacting to whichever order the system selects.

Product decision

This story expands the current station menu from a “what is needed here” guidance surface into an **explicit dish-start surface**. The current menu is useful because it highlights dishes with queued demand but ultimately routes through the existing generic cook action, which selects the oldest eligible ticket at that station. The new model must add a server-authoritative selected-dish cooking action so player intent is preserved.^[3]

Requirements

- Each station exposes the dishes it can legally produce.
- The menu displays:
 - Dish name and visual card
 - Station type
 - Estimated cook duration
 - Ingredient availability
 - Current queued demand
 - Current prepared/held count, if applicable
 - Whether the player is starting a demand-backed or speculative batch
- Dishes with active queued demand sort first and receive visual emphasis.
- Dishes with no active queue demand remain selectable if ingredients and station capacity permit.
- Selecting a dish sends an explicit server-authoritative request specifying the station and desired dish.
- The server validates:
 - Station supports the dish
 - Station capacity is available
 - Required ingredients are available or can be reserved
 - The requested action is permitted in the current phase
 - The player is within legal interaction range
- Starting a dish uses the same station-step timing model as existing kitchen production.
- The player does not need to remain at the station once production begins.
- A station cannot begin a conflicting item while already occupied unless the station supports explicit queue capacity.

Acceptance criteria

- A player can start a specific valid dish from the station menu.
- A demanded dish and an undemanded speculative dish are both selectable when legal.
- An unavailable dish clearly explains whether the block is ingredients, station occupancy, phase, or another authoritative reason.
- A started dish visibly transitions the station into an active cooking state.
- The cook timer continues after the initiating player walks away.
- Co-op teammates see the same station's active dish and remaining progress.
- Existing simple interact-to-cook behavior remains available as a fast path for the highest-priority eligible ticket, if retained.

API concept

```
type StartDishAction = {  
  type: "startDish";  
  stationId: StationId;  
  dishId: DishId;  
  intent: "demand" | "prep";  
};
```

The server resolves this to an existing order ticket when the intent is demand-backed and a matching queued ticket exists. Otherwise it creates or starts a prep-production job that uses the same legal station and inventory systems.

Story 4: Timed cooking and autonomous completion

User story

As a player who starts a dish, I want it to take an appropriate amount of time while I work elsewhere so that cooking feels like restaurant operations rather than an instant menu action.

Requirements

- All dishes have authored cooking duration based on their complexity and station steps.
- Dish complexity is communicated through estimated time and optional complexity markers, not hidden simulation values.
- A player-started dish follows the same underlying station timer and completion rules as a worker-started dish.
- The kitchen continues progressing while the player performs another action, moves across the restaurant, restocks, carries food, or interacts with management tools.
- The station presents distinct states:
 - Idle

- Waiting or queued
- Cooking
- Ready
- Blocked by ingredient shortage
- Temporarily unavailable
- Ready food follows the established service-pass or held-food workflow.
- The system must avoid duplicate completions or race conditions when both co-op players act around the same station.

Acceptance criteria

- Starting a dish creates an active timed state visible in the world and to both players.
- More complex dishes have longer authored production times than simple dishes.
- Leaving the station never cancels a valid started dish.
- A dish completes exactly once and enters the correct next state.
- A waiting-to-cook signal remains distinct from a long queue and from an ingredient shortage.
- Existing station indicators continue to follow the shared visual-state language. ^[4] ^[6]

Story 5: Speculative prep and held food

User story

As a strategic kitchen player, I want to cook dishes before customers order them so that I can prepare for a rush, event, promotion, sale, or anticipated demand pattern.

Requirements

- Players may start any legally cookable dish even when no matching active ticket exists.
- Speculative prep uses the same ingredient claims, station capacity, timing, and completion path as demand-backed production.
- Completed prep dishes enter a held-food inventory or service-pass-ready state that can satisfy a compatible future order.
- The kitchen display distinguishes:
 - Active demand-backed cooking
 - Active speculative prep
 - Prepared stock currently available
 - Prepared stock at risk of quality decay
- The game must not force a player to justify speculative prep against a current ticket.

- The system may apply hold-time decay, waste, freshness penalties, or an eventual discard state, but these rules must be explicit and visible.
- Promotions, events, and kitchen-command focus states may recommend prep opportunities but must not automatically start production.

Acceptance criteria

- A player can start a dish with zero current demand when ingredients and station capacity are available.
- The completed dish is visible in prepared inventory and can fulfill a later matching order.
- Prepped food cannot be mistaken for an active ticket unless it is assigned to one.
- A player receives clear feedback when prep expires, loses quality, or blocks needed capacity.
- Co-op teammates can tell that a dish was intentionally prepped rather than currently ordered.
- The results and telemetry systems can eventually explain waste or prep effectiveness, but no new results-screen work is required for the first implementation.

Balancing principles

Decision	Strategic upside	Trade-off
Pre-cook simple high-volume dishes	Faster fulfillment during sudden demand	Consumes ingredients and station time
Prep around an event or sale	Captures temporary demand opportunity	Risk if demand does not arrive
Hold premium dishes	Faster VIP or critic service	Higher waste or freshness downside
Fill every station early	Maximum apparent throughput	Less capacity to react to new orders

Story 6: Ingredient risk and quick restock

User story

As a kitchen player, I want to see ingredient risk before running out and quickly replenish critical stock so that I can prevent avoidable station blocks without opening the full pantry panel.

Requirements

- Show ingredient health in the kitchen view without requiring the pantry/restock panel.
- Use the established visual-state language:
 - Green: healthy
 - Yellow: attention soon
 - Orange: operational bottleneck

- Red: critical shortage or blocked production
- Keep shortage iconography visually distinct from queue depth and waiting-to-cook indicators.
- Surface a compact **Restock Critical** action when at least one relevant ingredient crosses the critical threshold.
- The quick action should default to restocking only ingredients required to resolve active or imminent production blocks.
- Before confirming, show:
 - Ingredients to replenish
 - Total cost
 - Delivery or availability timing, if applicable
 - Reason for the recommendation
- After restocking, display a short inline confirmation with the updated critical-state count.
- Preserve the full pantry interface for advanced selection, broader shopping, and cost control.

Acceptance criteria

- A player can identify low and critical stock while standing in the kitchen.
- A critical restock action is available without opening the pantry panel.
- A player cannot accidentally restock unavailable or unaffordable items without a clear error explanation.
- Restock outcomes synchronize for both co-op players.
- Quick restock does not bypass normal cost, inventory, delivery, or server validation rules.
- The scene differentiates a shortage from a long station queue through shape, position, and color—not color alone.^[6]

Story 7: Kitchen recommendations without forced play

User story

As a player, I want the kitchen to recommend useful work without preventing me from making my own production decisions so that the system helps during pressure while preserving strategy and experimentation.

Requirements

- The queue board and station menu may visually recommend dishes based on active queue, customer patience, known event pressure, ingredient risk, or kitchen focus.
- Recommendations must be qualitative, for example:
 - “Urgent”
 - “Good prep”
 - “Low stock risk”
 - “Event demand”
 - “Premium opportunity”
- The system must not expose raw utility scores or hidden customer preferences.
- Recommendations must not prevent selection of another valid dish.
- Kitchen-command focuses continue to bias worker prioritization, but they do not create food, inventory, or guaranteed outcomes.
- The recommendation system should make the active kitchen-command focus understandable in the local kitchen context.

Acceptance criteria

- A player can see why a recommended dish is recommended.
- A player can deliberately choose a non-recommended valid dish.
- The active kitchen focus is understandable at the kitchen board and relevant station menu.
- Recommendations do not compute game logic in the rendering layer. ^[2] ^[5]

Visual language

Required indicators

Indicator	Meaning	Primary visual treatment	Existing language
Queue stack	Work waiting at a station	Stacked blocks at station front	Yellow to orange by depth
Waiting-to-cook glyph	A queued dish has not started	Distinct station-top marker	Attention yellow
Active-cook state	A dish is being produced	Timer or animated cooking state	Orange bottleneck
Ready-food state	Completed dish awaits handoff	Pass or held-food marker	Green healthy
Ingredient shortage	A dish cannot proceed due to stock	Fixed shortage glyph at distinct station anchor	Red critical
Prep stock	Completed speculative food exists	Dish card or held-food badge	Blue/neutral prep state
Aging prep	Held food nearing quality loss	Time or freshness band	Yellow then orange

Indicator	Meaning	Primary visual treatment	Existing language
Restock critical	Urgent replenishment is available	Compact action strip with item badges	Red-to-brass urgency

The scene already has a documented requirement that important bottlenecks remain visible in the 3D world and that shortage icons differ from queue markers by geometry, color range, and anchor location. New indicators must build on that vocabulary rather than introduce arbitrary colors or ambiguous symbols.^[6]

Visual asset direction

New visual assets are not automatically required. The existing queue board uses programmatically rendered 2D dish cards rather than a hand-authored illustration set, and this is likely still the best first implementation path for station and restock indicators.^[1]

Generate new visual assets only when needed

Use Blender MCP when a visual requirement cannot be addressed through existing scene primitives, icon sprites, typed labels, dish cards, materials, or UI composition.

Potential Blender MCP asset candidates:

- A stronger physical frame, housing, or light treatment for the kitchen queue wall
- Distinct station-top appliances or signage where existing stations are difficult to identify
- A recognizable pantry or restock callout prop
- Held-food shelves, warming zones, or prep-rack props that communicate available prepared inventory
- New dish state models when a ready, held, aging, or spoiled state cannot be communicated clearly with existing food assets and color/material treatment

Blender MCP acceptance requirements

- Reuse existing production loaders and the established asset-catalog workflow.
- Match existing low-poly Copper & Thyme visual direction.
- Preserve correct scale, pivots, naming, materials, and transform conventions.
- Include any required state variants as documented metadata, not bespoke scene code.
- Validate assets in the asset showcase harness and in an in-context kitchen composition.
- Confirm legibility from the default and kitchen-operations cameras.
- Avoid visual-only state distinctions for critical gameplay information; pair assets with durable icon, color, or text signals.

The current project architecture includes a standalone harness ecosystem and a shared production scene path, so visual verification should happen outside a full multiplayer match as well as in the live restaurant context.^[5]

Technical architecture

Authority and responsibilities

Layer	Responsibilities
Server simulation	Dish eligibility, station capacity, ingredient claims, prep jobs, cook timers, completion, quality decay, restock validation, synchronization
Shared data	Dish-to-station mapping, cook durations, ingredient requirements, prep/hold rules, visual metadata, balance tuning
WebSocket snapshot	Per-viewer kitchen queue, station state, prepared inventory, relevant shortages, restock availability, active kitchen focus
React UI	Station menu, restock confirmation, kitchen-focused HUD, detailed inspection panels, accessibility and keyboard navigation
Three.js scene	Kitchen camera, station indicators, queue-board cards, held-food display, proximity affordances, world readability
Harnesses	Kitchen bottleneck, restaurant layout, asset showcase, mocked station/prep/shortage scenarios

This preserves the project’s core architecture: simulation belongs in registered server systems, while the client renders server-published state and does not manufacture authoritative game outcomes. ^[5] ^[6]

New or changed server concepts

- `startDish(stationId, dishId, intent)`
- `intent: "demand" | "prep"`
- Prepared-food or held-food records
- Optional freshness and expiration timestamps
- Station active-job records that identify whether work is demand-backed or speculative
- A quick-restock recommendation resolver that proposes valid restock items but never bypasses server checks

Suggested module boundaries

```
server/src/game/systems/order-system.js
server/src/game/systems/inventory-system.js
server/src/game/validators/action-validator.js
server/src/game/match.js

shared/game-data/dishes.json
shared/game-data/pantry-restock.json
shared/schemas/messages.d.ts
shared/schemas/game-state.d.ts
shared/constants/tuning.js
```

```
client/src/game/GameClient.ts
client/src/game/InteractionController.ts
client/src/scenes/RestaurantScene.ts
client/src/scenes/icon-sprites.ts
client/src/ui/StationMenu.tsx
client/src/ui/KitchenQueueBoard.tsx
client/src/ui/QuickRestock.tsx
client/src/app/GameView.tsx

harnesses/src/kitchen-bottleneck-harness.ts
harnesses/src/restaurant-layout-harness.ts
harnesses/src/asset-showcase-harness.ts
```

Telemetry

Track the following per match and per restaurant for balancing and player-behavior analysis:

- Total dishes started
- Dishes started from active order demand
- Dishes started as speculative prep
- Speculative dishes later served
- Speculative dishes wasted, expired, or discarded
- Time from ticket queue to cooking start
- Time from cooking completion to service
- Station utilization by station type
- Ingredient shortage incidents
- Quick-restock activations
- Restock prevented-block count
- Kitchen camera entries and time spent in kitchen mode
- Queue-board interaction and station-menu selection rate

Telemetry should be aggregated at meaningful lifecycle transitions, not written per simulation tick, consistent with the project's existing approach to durable and aggregate data. ^[5]

Validation plan

Harness coverage

- **Kitchen bottleneck harness:** Demand surge, station backlog, shortage, waiting-to-cook, prep inventory, active cooking, and ready-food states.
- **Restaurant-layout harness:** Kitchen camera framing, board readability, interaction range, and visual overlap validation.

- **Asset showcase harness:** Any Blender-generated board, station, pantry, or food-state asset in isolation and contextual presentation.
- **Mock visual-state fixture:** Side-by-side shortage, long queue, active cooking, prep availability, and urgent restock states.

End-to-end co-op test

1. Start a co-op service match.
2. Move one player into the kitchen and confirm camera transition.
3. Spawn or receive multiple dishes across stations.
4. Confirm board rank and station demand visibility.
5. Start a demanded dish and leave the station.
6. Start a speculative dish with no active matching ticket.
7. Force an ingredient-low and ingredient-empty state.
8. Use quick restock and confirm both clients synchronize.
9. Serve a later ticket using prepped food.
10. Verify aging, waste, or hold-quality behavior if enabled.
11. Run the normal full check, client build, harness build, and targeted server checks.

Rollout order

1. **Kitchen camera and readability pass** — operational framing, board visibility, station-state clarity.
2. **Explicit station dish selection** — add the authoritative specific-dish start action while preserving the current fast-path interaction.
3. **Prep and held-food model** — allow selected dishes to be produced without current demand.
4. **Quick restock surface** — expose inventory risk and urgent replenishment in kitchen context.
5. **Visual polish and optional Blender assets** — only after real gameplay verification identifies an unmet readability problem.
6. **Telemetry and balance tuning** — use speculative-prep effectiveness, shortage incidents, and kitchen utilization to tune hold windows, costs, and cook durations.

This order deliberately resolves perception and legibility first, then adds new decision-making power, then tunes the resulting strategic economy.

✱

1. [STORY-057-kitchen-command-position-and-expo-rail-wall.md](#)
2. [STORY-036-kitchen-command-board.md](#)
3. [STORY-042-coop-what-to-cook-menu.md](#)

4. [STORY-041-coop-timed-cooking-and-waiting-indicator.md](#)
5. [architecture.md](#)
6. [STORY-016-visual-state-language.md](#)
7. [STORY-043-coop-kitchen-order-queue-board.md](#)